Towards an Extensible Framework for Near Data Processing in AI:

Integration Experiences with oneAPI, OpenXLA, and PyTorch

Wonjae Lee, Vishnu Charan Thummala, Somnath Roy

Samsung Electronics Co., Ltd.



OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

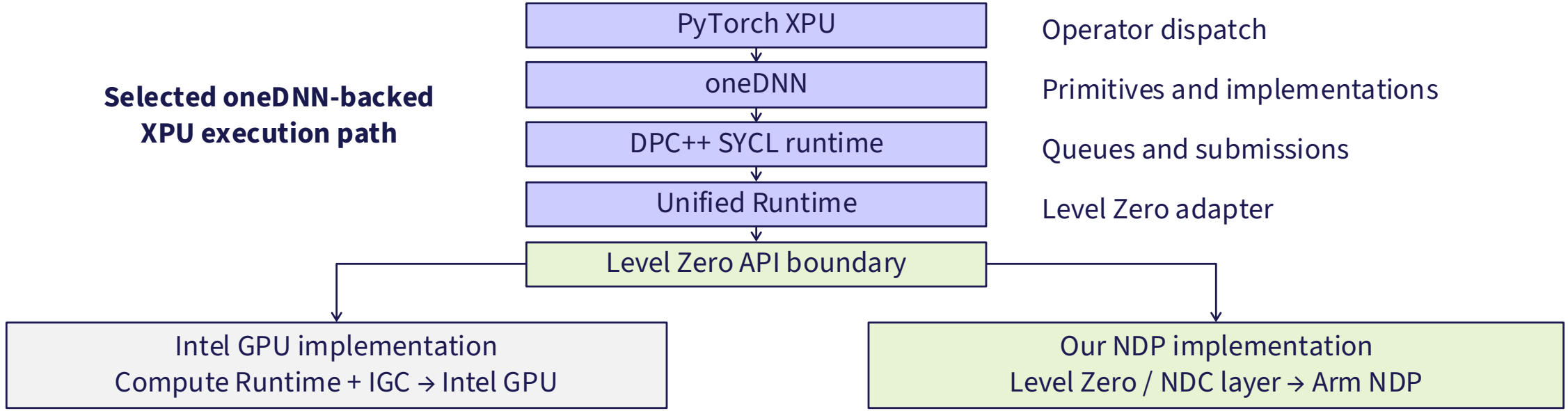
NDP Integration into AI Software Ecosystems

- **AI software ecosystems expose device integration at different layers.**
  - Three ecosystems expose different extension points for the same NDP integration goal.
  - Each boundary has a different implementation contract for the device vendor.

oneAPI · Level Zero	OpenXLA · PJRT C API	PyTorch · PrivateUse1
IMPLEMENT A LEVEL ZERO DRIVER	**IMPLEMENT A PJRT PLUGIN**	**BUILD AN OUT-OF-TREE BACKEND**
Integration interface: *Low-level device API for memory, kernels, and command submission*	***Integration interface:*** *Compilation and execution API with device and buffer management*	***Integration interface:*** *Operator dispatch and device-runtime integration hooks*
Vendor deliverable: *Level Zero driver backed by a device compiler/runtime*	***Vendor deliverable:*** *PJRT plugin backed by a device compiler/runtime*	***Vendor deliverable:*** *Out-of-tree backend with operator kernels and runtime integration*

oneAPI Integration: Feasibility PoC

- **XPU: PyTorch's built-in accelerator backend for Intel GPUs**
  - Applications use device="xpu" and torch.xpu; this path uses the oneAPI software stack.
  - The original XPU proposal covered Intel heterogeneous devices; current upstream enumeration is Intel GPU focused.
- **PoC: offloaded computation to an Arm-based CXL NDP device[2] using precompiled NDP kernels**
  - Level Zero entry points over NDC API[1], with Intel GPU properties emulated for PoC compatibility



1) OCP NDC API: https://github.com/opencomputeproject/samsung-ndc

2) 16 Arm Neoverse V2 cores, 4 DDR5 channels, CXL 2.0 interface.

Lessons and Next Steps

Integration findings: Supporting Arm-based NDP requires integration across multiple software layers

- **XPU scope**
  - Current PyTorch XPU implementation is Intel GPU-focused.
  - It enumerates Level Zero GPU devices and queries Intel GPU architecture.
- **oneDNN kernel path**
  - Intel GPU kernel paths may require OpenCL compilation and Intel-native binaries, even with Level Zero execution.
- **NDC runtime gap**
  - Kernel loading and launch are implemented;
  - SPIR-V-to-Arm compilation is missing.

Next steps under investigation: Define the NDP backend and kernel compilation path

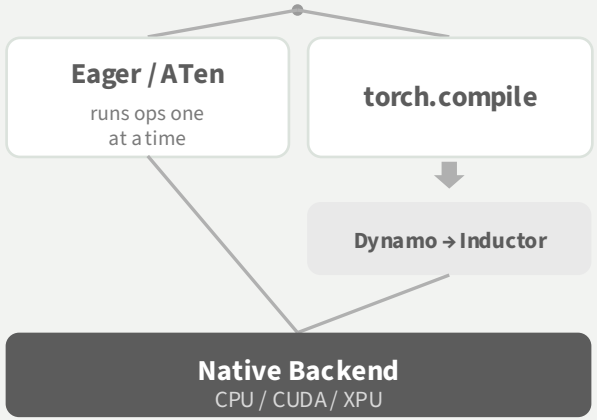
- **Translation**
  - Evaluate supported compute SPIR-V → LLVM IR via SPIRV-LLVM-Translator.
  - Investigate NDP-specific lowering, AArch64 code generation, and NDC runtime integration.
- **Backend exploration**
  - Evaluate a dedicated NDC backend as an alternative to the Intel GPU-focused XPU path.
  - Assess existing backend implementations for adaptation to NDP, including oneDNN's generic SYCL GPU kernels.

Why We Explored OpenXLA for NDC

A PyTorch workload can reach hardware through different software paths — our PoC took the OpenXLA route.

PyTorch LLM / Application

NATIVE PYTORCH PATHS



WHY EXPLORE OPENXLA?

- **Framework portability**
- **Graph-level compiler view**
- **Portable IR boundary**
- **Pluggable device runtime**

OPENXLA PATH — OUR PoC

- **PT** — PyTorch/XLA
- **HLO** — StableHLO / HLO
- **XLA** — XLA Compiler
- **PJ** — PJRT
- **NDC** — NDC PJRT Plugin → Runtime → CXL NDC

Can CXL NDC become another execution target beneath this Open XLA ecosystem?

OpenXLA in One Picture: What → Compile → Run

A simple mental model before we zoom into the PJRT-to-NDC mapping.

Framework frontends · PyTorch/XLA · JAX · TensorFlow

StableHLO / HLO
WHAT
Represent the computation

XLA Compiler
COMPILE
Optimize + lower for target

PJRT
MANAGE + EXECUTE
Uniform device/runtime API

ZOOM INTO THE PJRT CONTRACT

- **DEV — Device**: discover / describe target
- **MEM — Memory / Buffer**: place / access data
- **EXE — Executable**: compiled artifact
- **EVT — Execute / Event**: launch / async completion

maps to your NDC runtime

NDC PJRT Plugin → **NDC Runtime / API** → **CXL NDC**

PJRT provides the boundary that keeps NDC-specific runtime details behind the device plugin.

OPENXLA · PJRT

PJRT: The Device Contract We Used for NDC

Our plugin maps standard PJRT concepts onto the existing NDC runtime and CXL-attached device.

XLA Compiler → PJRT Interface → NDC PJRT Plugin → NDC Runtime / API → CXL NDC

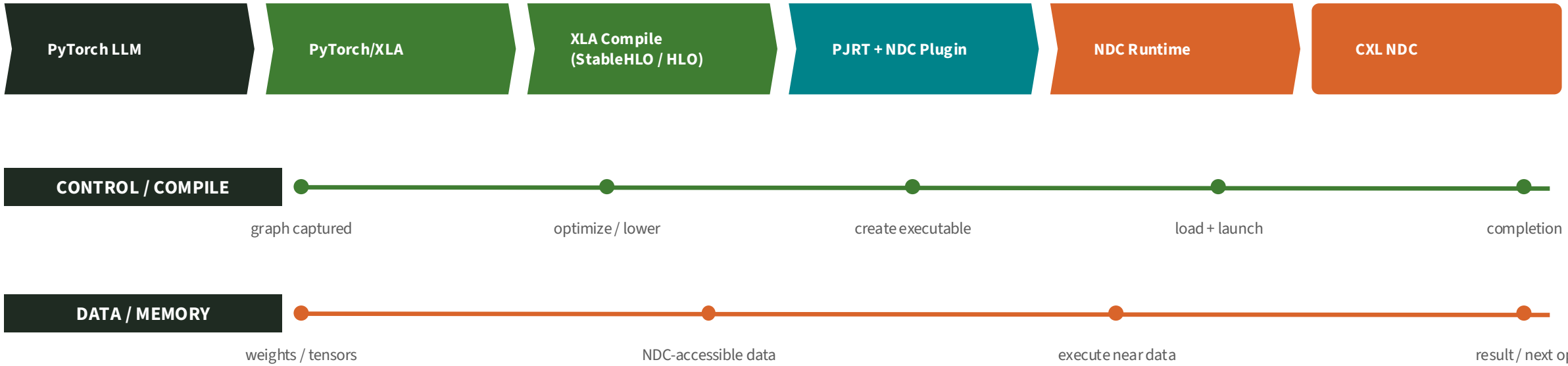
PJRT CONCEPT	PLUGIN MAPPING	NDC REALIZATION
DEV Device	PjRtDevice	NDC target + capabilities
MEM Memory / Buffer	PjRtBuffer / MemorySpace	CXL / NDC-accessible memory
EXE Executable	PjRtExecutable	NDC executable / operation
EVT Execute / Event	Execute + Future/Event	launch + completion

Architectural value: device-specific mechanisms stay below the PJRT plugin boundary.

One LLM Operation, End to End

Illustrative GEMV-style path: compile/control flow; data-memory path.

Example: linear projection / GEMV-style kernel — matrix × vector multiply during token-by-token inference



NDC value depends on preserving data locality: move the computation to the data — not large data back to the compute.

PoC validates the software path. Placement and locality determine the system-level benefit.

OpenXLA Integration: What Worked, What Remains

The PoC validated the NDC backend path. GPU + NDC execution introduces additional orchestration requirements.

WHAT THE PoC VALIDATED

Framework-agnostic backend path

PyTorch/XLA → OpenXLA
provides a path that is not tied to a single application framework.

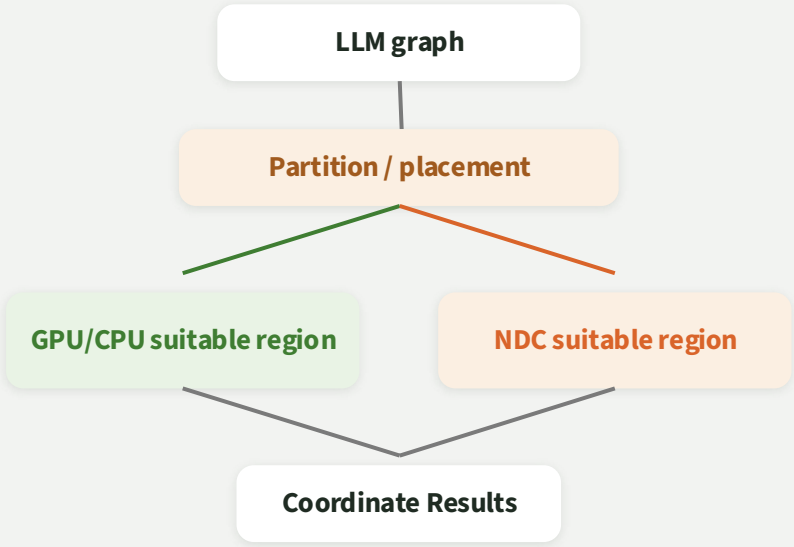
Clean device boundary

PJRT gives a natural point to expose NDC device, memory, executable and execution semantics.

End-to-end execution

We validated the path from framework/compiler through the NDC plugin/runtime to the CXL-attached device.

NDC AS A COMPLEMENTARY ACCELERATOR



The question changes:

“What should run where?”

WHAT REMAINS FOR GPU + NDC

Partitioning

Which graph regions belong on GPU vs NDC?

Data locality

Where should weights / activations live, and when should data move?

Cross-device Coordination

Dependencies, synchronization and execution of different target programs.

MPMD-style Orchestration

The heterogeneous execution model needed by our PoC was not mature enough in the path evaluated.

OpenXLA/PJRT validated the integration path; the subset-of-ops experience highlights additional requirements for an extensible NDC framework.

SPEL (Samsung Pytorch Extension Library)

Why OpenXLA + PJRT or SOTA Pytorch won't work optimally for our custom accelerators ?

Eager & graph execution

OpenXLA motivation is for whole graph compilation but we need both eager and graph mode execution. Also, for graph mode Pytorch ecosystem default to torch.inductor

Dynamic shapes

Variable seq-len and continuous batching mean padding, bucketing and recompiles with OpenXLA.
torch.compile worked much better for us

Ops and semantics

Thousands of ATen ops vs. a small HLO core; mutation and views need functionalization; numerics drift.

Lazy data placement

Accelerators with heterogeneous compute units can't afford to create input tensor's data storage early

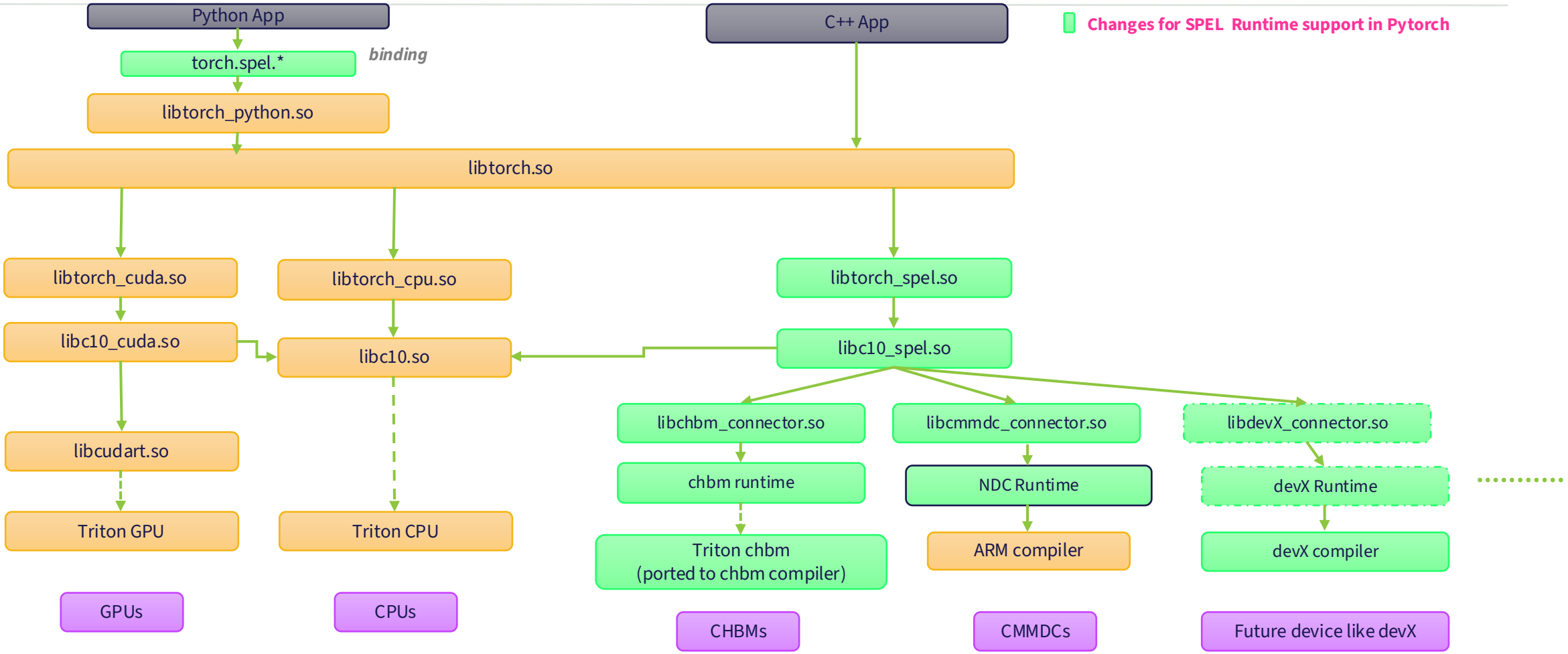
Limited shading support

Dtensors are still experimental. No way to dictate tensor's data placement and sharding across compute units

Multiple accelerator support

Supporting multiple accelerators of different characteristics with a single dispatch key

SPEL Architecture



SPEL Feature set

Pytorch native framework

- Full Eager mode support
- Only offloading to accelerator advertised operations, rest will fall back to CPU
- Accelerator vendors comes with runtime libraries and compilers (if needed) and this framework can enable them to run a model E2E
- Full integration with PT profiler

Lazy Tensor Creation

- SPEL support creating torch.tensors data storage lazily without changing PT programming interface
- Tensor data storage will be created just before operation is initiated on the tensor
- For accelerators having multiple Pus this will help to place tensor data optimally

Generic sharding support

- SPEL supports optional parameter for sharding hint during tensor creation
- SPEL also supports separate APIs before any op execution to let backend know about the execution plan including sharding
- PT programming interface will change by introducing new APIs or adding params to existing APIs

Pytorch native Graph mode support (WIP)

- SPEL can be integrated as dynamo backend for maximum flexibility
- SPEL optionally can also be integrated as a inductor backend with custom code generation logic
- Optimal data placement and sharding can be done by analyzing FX graph, may not need to use custom APIs

Qwen3 model run using SPEL + NDC on top of CMMDC accelerator device

```
Running qwen3-0.6B on spel:0
transformers.models.qwen3.modeling_qwen3
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
<think>

</think>
```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self SPEL	Self SPEL %	SPEL total	SPEL time avg	# of Calls
spel::mm	90.20%	154.066s	90.20%	154.066s	97.758ms	3.205s	100.00%	3.205s	2.034ms	1576
aten::empty	0.25%	430.510ms	0.25%	430.510ms	23.726us	0.000us	0.00%	0.000us	0.000us	18145
aten::to	0.02%	38.556ms	4.41%	7.537s	211.113us	0.000us	0.00%	0.000us	0.000us	35703
aten::_to_copy	0.05%	88.918ms	4.39%	7.499s	249.237us	0.000us	0.00%	0.000us	0.000us	30087
aten::empty_strided	0.06%	102.185ms	0.06%	102.185ms	3.213us	0.000us	0.00%	0.000us	0.000us	31807
aten::copy_	0.04%	70.425ms	5.68%	9.696s	316.430us	0.000us	0.00%	0.000us	0.000us	30643
aten::_copy_from	5.93%	10.123s	5.93%	10.136s	309.907us	0.000us	0.00%	0.000us	0.000us	32705

```
content: The capital of France is **Paris**
```

Code will be open sourced here by end of OCP:

http://github.com/SamsungDS/Cognos/SPEL

Thank You!

OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

